NEO-6M GPS Module

Hardware Overview

What is it?

The **NEO-6M GPS Module** is a GPS-receiver that picks up radio signals from satellites orbiting the Earth. It gives you the exact Location (Latitude/Longitude), Altitude, Speed, and Time.

Electrical Explanation

The module contains a tiny ceramic antenna and a high-sensitivity radio receiver.

- **Triangulation:** It listens for timestamps sent by satellites. By calculating how long the signal took to reach the antenna (time of flight) from at least 3 satellites, it calculates your position.
- **Communication:** It sends data to the microcontroller using Serial Communication. It "speaks" a language called NMEA, which the library translates into numbers.

Key Hardware Facts:

- **RX Pin:** The K-Comp receives data on for example Digital Pin **D6**.
- **TX Pin:** The K-Comp sends commands on for example Digital Pin **D7**.
- **Voltage:** Operates at **3.3V to 5V**.
- **Signal Flow:** Satellites ➔ Antenna ➔ GPS Chip ➔ Serial Data ➔ K-Comp Processor.

Software & Programming

kcomp_NEO6MGPS.h

Parsing the raw text from a GPS device is difficult. This library handles reading, parsing, and math.

Usage: `#include <kcomp_NEO6MGPS.h>`

The Data Structure

Unlike other sensors that give you a single number (like `int`), the GPS gives you a `struct` (a package of variables). When you call `getGPSData()`, you get a package containing all of this:

Field	Type	Description
valid	**bool**	`true` if the GPS has a "fix" (locked onto satellites).
latitude	**double**	Your `N/S` position (e.g., `48.2082`).
longitude	**double**	Your `E/W` position (e.g., `16.3738`).
speed	**double**	Current speed in `km/h`.
altitude	**double**	Current height in `m`.
year, month, day	**int**	Current date (automatically adjusted for timezone).
hour, minute	**int**	Current time (automatically adjusted for timezone).

Function Reference

Function Name	Description & Parameters
bool initNEO6MGPS(rx, tx, baud)	**Setup:** Starts the module. ▪ `rx/tx`: Pins (usually `6` and `7`).

	▪ baud: Speed (usually 9600). **Returns:** true if successful.
bool updateGPS()	**The Worker:** Reads incoming data from the serial port. **Important:** Must be called *constantly* in loop() to keep the data fresh.
GPSData getGPSData()	**Read:** Returns the GPSData struct described above.
void setGPSOffset(int hours)	**Config:** Sets your time zone offset (e.g., 1 for Central Europe).
void setGPSDST(bool enabled)	**Config:** Set to true if summer time is active.
double gps_distanceBetween(...)	**Math:** Calculates distance in **meters** between two coordinates (lat1/lon1 and lat2/lon2).
GPSCardinal gps_cardinal(deg)	**Math:** Converts a heading (0-360) into a direction (e.g., CARDINAL_N, CARDINAL_SW).

⚠️ Pitfalls & Troubleshooting

- **The "Cold Start" (Patience!)**
  After powerup the module might need 1 to 5 minutes to get its first "fix" on the position.
  The Fix: Go close to a window! GPS signals **cannot** penetrate buildings.
- **2. Blocking Code Kills GPS**
  The GPS sends data constantly. If your code stops (e.g., using delay(2000)), the serial buffer will overflow, and you will lose data.
  The Fix: Never use delay() in your loop. See the example below.

Sample Program

This program displays your location.

```
#include <kcomp_NEO6MGPS.h>

void setup() {
  Serial.begin(115200);

  if (!initNEO6MGPS(6, 7, 9600)) { // 1. Init. GPS on Pins 6 (RX) and 7 (TX)
    Serial.println("GPS Error: Check wiring!");
    while(1);
  }

  setGPSOffset(1);  // 2. Configure Timezone (e.g., UTC+1 for Vienna/Berlin)
  setGPSDST(false); // Set true in Summer

  Serial.println("Waiting for Satellite Fix (Go Outside!)...");
}

void loop() {
  // --- Step 1: Keep the GPS "Fed" ---
  // We want to print every 1 second, but we cannot sleep!
  // We loop for 1000ms, constantly calling updateGPS().
  unsigned long start = millis();
  while (millis() - start < 1000) {
    updateGPS();
  }

  // --- Step 2: Read Data ---
```

```
  GPSData gps = getGPSData();

  if (gps.valid) {
    // Print Location
    Serial.print("Lat: "); Serial.print(gps.latitude, 6);
    Serial.print(" | Lon: "); Serial.println(gps.longitude, 6);
  } else {
    Serial.print("Searching... Satellites visible: ");
    Serial.println(gps.satellites);
  }
}
```

💡 **Try it out:** You can find the full sample program **`SimpleNEO6MGPS`** in the K-Comp examples folder!